Build Touch Applications

TOUCH FOR DESKTOP APPS

First of all, let us clarify desktop applications here doesn't just mean applications developed for non-portable traditional desktop machines. What we mean to convey is that these are applications that are delivered via the traditional desktop model. The user downloads or otherwise acquires the software, and then installs it. The software runs directly on the machine rather than in a web browser. The machine itself could be a desktop, notebook or netbook. In fact what we are talking about is more relevant to portable devices with touch screens, not actual desktops.

Looking back at the way the research subjects approached touch capability, they tended to move between touch and the mouse depending on subtle factors in addition to conscious awareness. What this tells us is that the most comfortable mode of operation is not one where we are insisting that they use one method or another; rather, it is indeterminate. A user may click a button on a form using the mouse 80 percent of the time and use touch 20 percent of the time one day and entirely reverse this pattern the following day.

Whatever you design has to suit either input mode. Do not decide for the user, or force the user to use only touch for certain tasks and only the mouse for other task. Of course, if you fail in your design and end up making the user's input decision for him or her, frustration will certainly ensue.

Incidentally, the choice of platform and framework used in the application's construction is probably not relevant when talking about touch optimisation. For example, Java and Microsoft's dot NET are equally capable of supporting developers introducing touch in that neither platform considers touch a first-class way of providing input into an application. Over time though, this view is likely to change as touch works its way backwards from post-PC devices to PC hardware. Microsoft's Metro for example, is entirely touch based and focused. Hence, any work you have to do you'll have to do yourself, although such work tends not to be onerous.

Really, all you're trying to do is be sympathetic to the user. After all if they are frustrated by using your application on a touch device they will look for an alternative that works. And the chances of this—that is a person using your device on a computer—are only going to increase.

TARGETS

For desktop applications, the major consideration is target size. Many business applications are form based, and such, applications tend to have a reputation for being tightly packed. Often the interfaces are dense with information and controls to improve the efficiency for repetitive tasks.

AS A DEVELOPER, WHY LIMIT YOUR TARGET AUDIENCE TO SMARTPHONE AND TABLET USERS? WITH WINDOWS 8, YOU CAN NOW TARGET INTEL ULTRABOOK USERS WITH RICH AND INTERACTIVE APPS THAT TAKE ADVANTAGE OF INBUILT SENSORS INCLUDING TOUCH AND PROXIMITY

Touch simply does not give the kind of precision that one can get with a mouse. While a mouse always registers an exact, single-pixel coordinate under the hotspot of the cursor, the operating system assumes the input from a touch screen is likely to be inaccurate and compensates accordingly. The operating system thus has to be a little intuitive and "guess" where you wanted to click based on where the user elements lie on the user interface, and the "fuzzy" area covered by the touch of a finger.

For example, if the touch screen reports that you pressed just outside of a field, the operating system may choose to interpret this as a click in the field, as you are more likely to want to click in the field than you are to click outside of the field. Therefore, one way in which you can help the operating system is to space the fields so that presses outside of fields can be more reliably mapped as intentions to click in fields. The same applies to buttons.

Another way to help the operating system is to make the fields larger, for the simple reason that a larger field is easier to hit. Generally though, the standard presentation of buttons and fields is of an appropriate size for touch or mouse operation.

Oftentimes, forms get cramped, because software engineers want to get everything onto a single form; so, it's easy to find applications that have fussy or fiddly forms. When touch-optimising this sort of form, you may find it appropriate to break out individual functionality into tabs. Of course, you also have to make the tabs an appropriate size. By default, tabs are typically too small, probably because designers generally considered them a secondary UI element. For touch optimisation, you're turning that secondary element back into a primary element by making it larger.

Toolbars and menus are another consideration. Menus are usually managed by the operating system, and as such, you can assume that the operating system will interpret touch in an optimal way when working with